

```

import {
  addCurso,
  getCursosCollection,
  deleteCursoCollection,
  getCursoCollection,
  updateCursoCollection,
  getPreceptoresCollection,
} from "../firebase.js";

/**
 * Función para levantar Venta Modal
 */
window.miModal = async function (idModal, idCurso = "") {
  try {
    await validarModal(idModal);

    let url = "";
    switch (idModal) {
      case "agregarCursoModal":
        url = "modales/modalAddCurso.php";
        break;
      case "detalleCursoModal":
        url = "modales/modalDetallesCurso.php";
        break;
      case "editarCursoModal":
        url = "modales/modalEditarCurso.php";
        break;
      case "eliminarCursoModal":
        url = "modales/modalDeleteCurso.php";
        break;
      default:
        throw new Error(`El idModal '${idModal}' no es válido`);
    }

    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Error al cargar la modal");
    }

    // response.text() es un método en programación que se utiliza para obtener el
    // contenido de texto de una respuesta HTTP
    const data = await response.text();

    // Crear un elemento div para almacenar el contenido de la modal
    const modalContainer = document.createElement("div");
    modalContainer.innerHTML = data;

    // Agregar la modal al documento actual
    document.body.appendChild(modalContainer);
  }
};

```

```

    // Mostrar la modal
    const modalElement = modalContainer.querySelector(`#${idModal}`);
    const myModal = new bootstrap.Modal(modalElement);
    myModal.show();
    if (idModal === "agregarCursoModal") { // Ajusta al ID real de tu modal de curso
        await cargarPreceptoresEnModal();
    } else if (idModal === "agregaralumnoModal") {
        await cargarCursosEnModal();
    }
    if (idModal === "detalleCursoModal") {
        await cargarDetalleCurso(idCurso);
    } else if (idModal === "editarCursoModal") {
        await getCursoUpdateCollection(idCurso);
    } else if (idModal === "eliminarCursoModal") {
        let DeleteBtn = document.querySelector("#confirmDeleteBtn");
        DeleteBtn.addEventListener("click", async () => {
            await eliminarCurso(idCurso);
        });
    }
    } catch (error) {
        console.error(error);
    }
};

```

```

//Función para validar si existe una modal abierta
async function validarModal(idModal) {
    const existingModal = document.getElementById(idModal);
    if (existingModal) {
        const modal = bootstrap.Modal.getInstance(existingModal);
        if (modal) {
            modal.hide();
        }
        existingModal.remove();
    }
}

```

```

/**
 * Función para obtener todas las colecciones
 */
async function mostrarCursosEnHTML() {
    try {
        const snapshot = await getCursosCollection(); // Llamada a RTDB
        const tablaCursos = document.querySelector("#tablaCursos tbody");
        tablaCursos.innerHTML = "";

        if (snapshot.exists()) {
            // RTDB usa forEach sobre el snapshot directamente
            snapshot.forEach((childSnapshot) => {
                const id = childSnapshot.key; // El ID generado por push()
            });
        }
    } catch (error) {
        console.error(error);
    }
}

```

```

        const Curso = childSnapshot.val(); // AQUÍ: usamos .val() en lugar de
.data()

        const fila = document.createElement("tr");
        fila.id = id;
        fila.innerHTML = `
            <td>${Curso.curso || "N/A"}</td>
            <td>${Curso.ubicacion || "N/A"}</td>
            <td>${Curso.capacidad || "N/A"}</td>
            <td>${Curso.obs || "N/A"}</td>
            <td>
                <button onclick="window.miModal('detalleCursoModal','${id}')"
class="btn btn-success"><i class="bi bi-binoculars"></i></button>
                <button onclick="window.miModal('editarCursoModal','${id}')" class="btn
btn-warning"><i class="bi bi-pencil-square"></i></button>
                <button onclick="window.miModal('eliminarCursoModal','${id}')"
class="btn btn-danger"><i class="bi bi-trash"></i></button>
            </td>
        `;
        tablaCursos.appendChild(fila);
    });
}
} catch (error) {
    console.error("Error al obtener los Cursos:", error);
}
}

window.addEventListener("DOMContentLoaded", mostrarCursosEnHTML);

/**
 * Función para agregar un nuevo Curso
 */

window.addNuevoCurso = async function (event) {
    event.preventDefault();

    // 1. Usamos el ID exacto de tu HTML: "formularioCursoEdit"
    const formulario = document.querySelector("#formularioCursoEdit");
    // const preceptorId = selectPreceptor.value;

    if (!formulario) {
        console.error("No se encontró el formulario #formularioCursoEdit");
        return;
    }

    const formData = new FormData(formulario);
    const formDataJSON = {};
    formData.forEach((value, key) => {
        formDataJSON[key] = value;
    });

```

```

const selectPreceptor = document.getElementById("selectPreceptorModal");
// 2. Extraemos los valores según los "name" de tus inputs
const { curso, ubicacion, capacidad, obsCurso, preceptorId } = formDataJSON;
if (!preceptorId) {
  alert("Por favor, seleccione un preceptor");
  return;
}
try {
  // 3. Llamamos a addCurso (la función de tu firebase.js)
  // Pasamos obsCurso al parámetro de observaciones
  await addCurso(curso, ubicacion, capacidad, obsCurso, preceptorId);

  // 4. Limpiar formulario y cerrar modal
  formulario.reset();
  $("#agregarCursoModal").modal("hide");

  // 5. Actualizar la tabla automáticamente
  if (typeof mostrarCursosEnHTML === "function") {
    await mostrarCursosEnHTML();
  }

  window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso creado con éxito!" });
} catch (error) {
  console.error("Error al crear el curso:", error);
  window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al crear el curso" });
}
};

```

```

/**
 * Función para cargar y mostrar los detalles del Curso en la modal
 */
async function cargarDetalleCurso(id) {
  try {
    const snapshot = await getCursoCollection(id); // Calling the fixed function above

    if (snapshot.exists()) {
      const CursoData = snapshot.val(); // USE .val() instead of .data()
      const { curso, ubicacion, capacidad, obs } = CursoData;

      const ulDetalleCurso = document.querySelector("#detalleCursoContenido ul");
      ulDetalleCurso.innerHTML = `
        <li class="list-group-item"><b>Curso:    </b> ${curso    || "No
disponible"}</li>

```

```

        <li class="list-group-item"><b>Ubicación:</b> ${ubicacion || "No
disponible"}</li>
        <li class="list-group-item"><b>Capacidad:</b> ${capacidad || "No
disponible"}</li>
        <li class="list-group-item"><b>Obs:      </b> ${obs      || "No
disponible"}</li>
    `;
    } else {
        console.log("No se encontró ningún Curso con el ID:", id);
    }
} catch (error) {
    console.error("Error al mostrar detalles del Curso", error);
}
}

```

```

/**
 * Buscar Curso a editar
 */
async function getCursoUpdateCollection(id) {
    try {
        const CursoDoc = await getCursoCollection(id);
        if (CursoDoc.exists()) {
            const CursoData = CursoDoc.val();
            const { curso, ubicacion, capacidad, obs } = CursoData;
            document.querySelector("#idCurso").value = id;
            document.querySelector("#curso").value = curso;
            document.querySelector("#ubicacion").value = ubicacion;
            document.querySelector("#capacidad").value = capacidad;
            document.querySelector("#obsCurso").value = obs;
        } else {
            console.log("No se encontró ningún Curso con el ID:", id);
        }
    } catch (error) {
        console.error("Error al obtener los detalles del Curso:", error);
    }
}

```

```

/**
 * Función para actualizar el Curso
 */
window.actualizarCurso = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioCursoEdit");
    const formData = new FormData(formulario);

    // Convertir FormData a un objeto JSON
    const formDataJSON = {};
    formData.forEach((value, key) => {
        //console.log(key, value);
    });
}

```

```

    formDataJSON[key] = value;
  });

  const { idCurso, curso, ubicacion, capacidad, obsCurso } = formDataJSON;
  try {
    await updateCursoCollection(idCurso, { curso, ubicacion, capacidad,
obs:obsCurso});
    formulario.reset();

    setTimeout(() => {
      $("#editarCursoModal").css("opacity", "");
      $("#editarCursoModal").modal("hide");
    }, 300);

    window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso actualizado
correctamente!" });
    await mostrarCursosEnHTML();
  } catch (error) {
    console.log(error);
  }
};

```

```

/**
 * Función para borrar un Curso, una colleccion
 */
async function eliminarCurso(id) {
  try {
    await deleteCursoCollection(id);
    document.querySelector(`#${id}`).remove();
    mostrarAlerta({ tipoToast: "success", mensaje: "Curso eliminado correctamente"
});
  } catch (error) {
    console.error("Error al borrar el Curso:", error);
    mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar el Curso" });
  }
}

```

```

/**
 * Función para mostrar alertas
 */
iziToast.settings({
  timeout: 10000,
  resetOnHover: true,

```

```

    // icon: '', // icon class
    transitionIn: "flipInX",
    transitionOut: "flipOutX",
    position: "topRight", // bottomRight, bottomLeft, topRight, topLeft, topCenter,
bottomCenter, center
    onOpening: function () {
        console.log("Alerta abierta!");
    },
    onClosing: function () {
        console.log("Alerta cerrada!");
    },
});
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (tipoToast == "success") {
        iziToast.success({
            timeout: 5000,
            icon: tipoToast == "success" ? "bi bi-check-circle-fill" : "bi
bi-x-circle-fill",
            title: tipoToast == "success" ? "¡Éxito!" : "¡Error!",
            message: mensaje,
        });
    } else if (tipoToast == "warning") {
        iziToast.success({
            timeout: 5000,
            icon: tipo == "success" ? "bi bi-check-circle-fill" : "bi bi-x-circle-fill",
            title: tipo == "success" ? "¡Éxito!" : "¡Error!",
            message: mensaje,
        });
    }
};

async function cargarPreceptoresEnModal() {
    const select = document.getElementById("selectPreceptorModal");
    if (!select) return;

    try {
        const preceptorSnap = await getPreceptoresCollection();
        select.innerHTML = '<option selected value="">Selecione
Preceptor</option>';

        if (preceptorSnap.exists()) {
            preceptorSnap.forEach(doc => {
                const data = doc.val();
                const option = document.createElement("option");

                // IMPORTANTE: El value ahora es el ID (la clave de Firebase)
                option.value = doc.key;
                // Lo que ve el usuario sigue siendo el nombre
                option.textContent = data.nombre;
            });
        }
    } catch (error) {
        console.error("Error al cargar preceptores: ", error);
    }
}

```

```
        select.appendChild(option);
    });
}
} catch (error) {
    console.error("Error:", error);
}
}
```